



Speeding-Up LULESH on HPX

Useful Tricks and Lessons Learned using a Many-Task-Based Approach

Torben Kalkhof Andreas Koch

2024-11-17

Embedded Systems and Applications, TU Darmstadt, Germany

Motivation

- Rising intra- and inter-node parallelism in modern supercomputers



https://pics.computerbase.de/1/0/8/4/7/5-0b304413bb1f8b62/7-2160_a4bf1e21.jpg

Motivation

- Rising intra- and inter-node parallelism in modern supercomputers
- But we are still stuck with the OpenMP/MPI programming model
 - Synchronization barriers between threads
 - Communication overhead between nodes
 - Unable to exploit available parallelism



https://pics.computerbase.de/1/0/8/4/7/5-0b304413bb1f8b62/7-2160_a4bf1e21.jpg

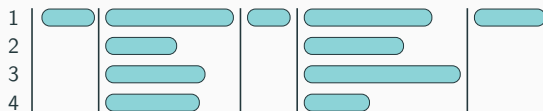
- Asynchronous Many-Task runtimes (AMTs) offer alternative model
 - Break down computation in countless fine-grained tasks and specify task dependencies
 - Runtime schedules tasks dynamically on available resources
 - Remove barriers and hide communication latency

- Asynchronous Many-Task runtimes (AMTs) offer alternative model
 - Break down computation in countless fine-grained tasks and specify task dependencies
 - Runtime schedules tasks dynamically on available resources
 - Remove barriers and hide communication latency
- Improved overall resource utilization

Motivation

- Asynchronous Many-Task runtimes (AMTs) offer alternative model
 - Break down computation in countless fine-grained tasks and specify task dependencies
 - Runtime schedules tasks dynamically on available resources
 - Remove barriers and hide communication latency
- Improved overall resource utilization

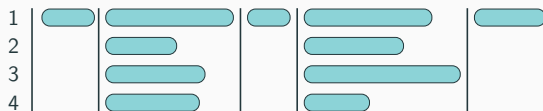
Fork-join with load imbalance:



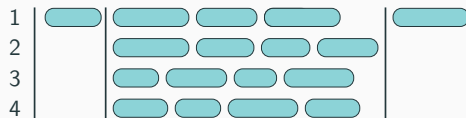
Motivation

- Asynchronous Many-Task runtimes (AMTs) offer alternative model
 - Break down computation in countless fine-grained tasks and specify task dependencies
 - Runtime schedules tasks dynamically on available resources
 - Remove barriers and hide communication latency
- Improved overall resource utilization

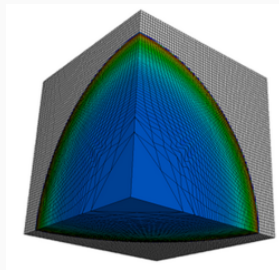
Fork-join with load imbalance:



Task-based programming model:



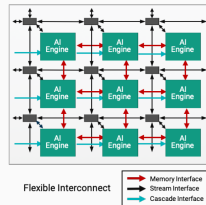
- Optimized many-task-based implementation of the LULESH proxy application using the HPX programming framework (single node)
- Provide details on conducted optimization steps
- Performance comparison to OpenMP-based reference implementation



https://asc.llnl.gov/sites/asc/files/2021-01/142_1-249x240.png

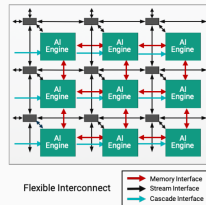
Background

- Enabling of FPGA tasks in HPX using TaPaSCo @ ARC '24
 - TaPaSCo: Open-source toolflow to integrate FPGA-based accelerators in heterogeneous systems
 - LULESH proxy application used for case study
 - Accelerator based on AMD AI Engines
 - Focus on TaPaSCo-HPX integration



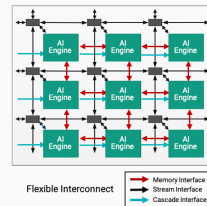
Background

- Enabling of FPGA tasks in HPX using TaPaSCo @ ARC '24
 - TaPaSCo: Open-source toolflow to integrate FPGA-based accelerators in heterogeneous systems
 - LULESH proxy application used for case study
 - Accelerator based on AMD AI Engines
 - Focus on TaPaSCo-HPX integration
- Porting effort to obtain HPX-based software baseline faster than OpenMP implementation



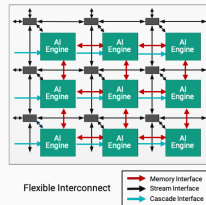
Background

- Enabling of FPGA tasks in HPX using TaPaSCo @ ARC '24
 - TaPaSCo: Open-source toolflow to integrate FPGA-based accelerators in heterogeneous systems
 - LULESH proxy application used for case study
 - Accelerator based on AMD AI Engines
 - Focus on TaPaSCo-HPX integration
- Porting effort to obtain HPX-based software baseline faster than OpenMP implementation
- Further optimizations for this work



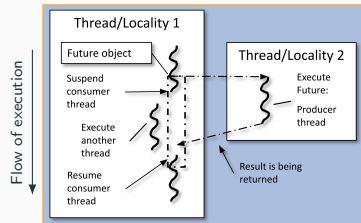
Background

- Enabling of FPGA tasks in HPX using TaPaSCo @ ARC '24
 - TaPaSCo: Open-source toolflow to integrate FPGA-based accelerators in heterogeneous systems
 - LULESH proxy application used for case study
 - Accelerator based on AMD AI Engines
 - Focus on TaPaSCo-HPX integration
- Porting effort to obtain HPX-based software baseline faster than OpenMP implementation
- Further optimizations for this work
- Share insights and lessons learned



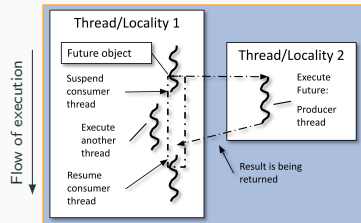
HPX Programming Framework

- C++ library based on concept of *futurization*
 - Future objects hold state and result of asynchronous operations
 - Build task graph using futures and continuations



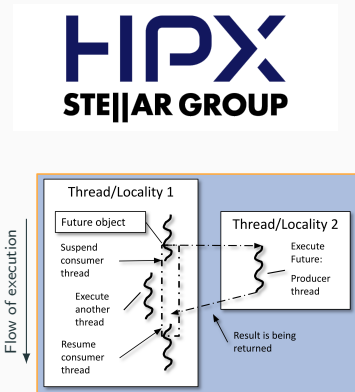
HPX Programming Framework

- C++ library based on concept of *futurization*
 - Future objects hold state and result of asynchronous operations
 - Build task graph using futures and continuations
- HPX API conforms to C++ standard library for concurrency and parallelism
 - `std::future<>` $\Leftrightarrow$ `hpx::future<>`
 - `std::async()` $\Leftrightarrow$ `hpx::async()`



HPX Programming Framework

- C++ library based on concept of *futurization*
 - Future objects hold state and result of asynchronous operations
 - Build task graph using futures and continuations
- HPX API conforms to C++ standard library for concurrency and parallelism
 - `std::future<>` $\Leftrightarrow$ `hpx::future<>`
 - `std::async()` $\Leftrightarrow$ `hpx::async()`
- Tasks implemented as light-weight user space threads
- Fixed number of OS threads to execute tasks
 - No expensive context switches on OS level



HPX Code Example

```
1  // create task (executed asynchronously)
2  hpx::future<int> f1 = hpx::async(do_some_work, 42)
3
4  // continuation executed after previous task completed
5  hpx::future<int> f2 = f1.then([](hpx::future<int> &&f_move) {
6      return do_more_work(f_move.get());
7  });
8
9  // create more tasks
10
11 // synchronization barrier
12 std::vector<hpx::future<int>> vec;
13 vec.push_back(std::move(f2));
14 hpx::wait_all(vec);
```

HPX Code Example

```
1  // create task (executed asynchronously)
2  hpx::future<int> f1 = hpx::async(do_some_work, 42)
3
4  // continuation executed after previous task completed
5  hpx::future<int> f2 = f1.then([](hpx::future<int> &&f_move) {
6      return do_more_work(f_move.get());
7  });
8
9  // create more tasks
10
11 // synchronization barrier
12 std::vector<hpx::future<int>> vec;
13 vec.push_back(std::move(f2));
14 hpx::wait_all(vec);
```

HPX Code Example

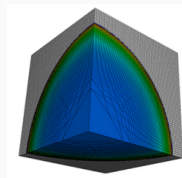
```
1  // create task (executed asynchronously)
2  hpx::future<int> f1 = hpx::async(do_some_work, 42)
3
4  // continuation executed after previous task completed
5  hpx::future<int> f2 = f1.then([](hpx::future<int> &&f_move) {
6      return do_more_work(f_move.get());
7  });
8
9  // create more tasks
10
11 // synchronization barrier
12 std::vector<hpx::future<int>> vec;
13 vec.push_back(std::move(f2));
14 hpx::wait_all(vec);
```

HPX Code Example

```
1  // create task (executed asynchronously)
2  hpx::future<int> f1 = hpx::async(do_some_work, 42)
3
4  // continuation executed after previous task completed
5  hpx::future<int> f2 = f1.then([](hpx::future<int> &&f_move) {
6      return do_more_work(f_move.get());
7  });
8
9  // create more tasks
10
11 // synchronization barrier
12 std::vector<hpx::future<int>> vec;
13 vec.push_back(std::move(f2));
14 hpx::future<...> f3 = hpx::when_all(vec);
```

LULESH Proxy Application

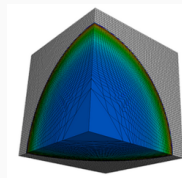
- Solves the Sedov Blast Wave Problem using Lagrange Hydrodynamics
- Iterative computation of forces and material properties in a hexahedral mesh



OpenMP

LULESH Proxy Application

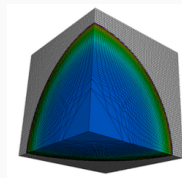
- Solves the Sedov Blast Wave Problem using Lagrange Hydrodynamics
- Iterative computation of forces and material properties in a hexahedral mesh
- Mesh divided into regions to mimic different material properties
 - Repeat calculation on more “expensive” regions



OpenMP

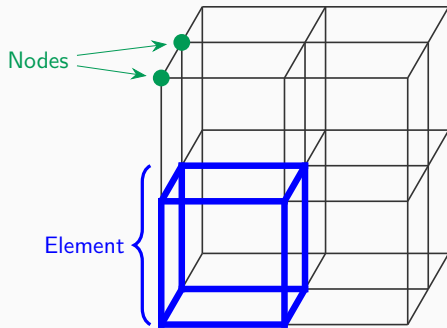
LULESH Proxy Application

- Solves the Sedov Blast Wave Problem using Lagrange Hydrodynamics
- Iterative computation of forces and material properties in a hexahedral mesh
- Mesh divided into regions to mimic different material properties
 - Repeat calculation on more “expensive” regions
- Reference implementation consists of 45 OpenMP parallel for-loops



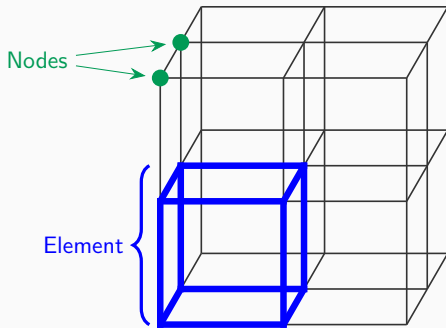
OpenMP

LULESH Proxy Application



- Lagrange Leapfrog algorithm with three phases:

LULESH Proxy Application

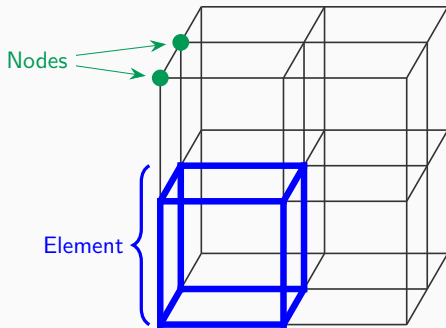


- Lagrange Leapfrog algorithm with three phases:

1. Advance nodal properties

- Calculate forces on nodes
- Node acceleration, position and velocity

LULESH Proxy Application



- Lagrange Leapfrog algorithm with three phases:

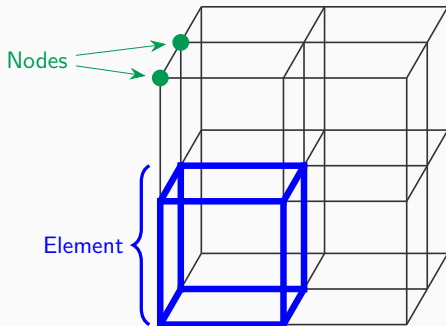
1. Advance nodal properties

- Calculate forces on nodes
- Node acceleration, position and velocity

2. Advance element properties

- Pressure, energy, volume, etc.
- Independent calculation for mesh regions

LULESH Proxy Application



- Lagrange Leapfrog algorithm with three phases:

1. Advance nodal properties

- Calculate forces on nodes
- Node acceleration, position and velocity

2. Advance element properties

- Pressure, energy, volume, etc.
- Independent calculation for mesh regions

3. Calculate next timestep

Structure of LULESH

- LULESH has the typical structure of OpenMP applications consisting of consecutive parallel for-loops
- Implicit synchronization barriers at the end of each loop

```
1  #pragma omp parallel for
2  for (i = 0; i < N; i++) { ... }
3
4  #pragma omp parallel for
5  for (i = 0; i < N; i++) { ... }
6
7  #pragma omp parallel for
8  for (i = 0; i < N; i++) { ... }
9
10 #pragma omp parallel for
11 for (i = 0; i < N; i++) { ... }
```

Prior Work

- Prior work: Not sufficient to simply use `hpx::for_each` constructs

```
1  hpx::for_each_n(execution::par, ... , [](int i) {
2      ...
3  });
4
5  hpx::for_each_n(execution::par, ... , [](int i) {
6      ...
7  });
8
9  hpx::for_each_n(execution::par, ... , [](int i) {
10     ...
11 });
12
13 hpx::for_each_n(execution::par, ... , [](int i) {
14     ...
15 });
```

Prior Work

- Prior work: Not sufficient to simply use `hpx::for_each` constructs
- Well-balanced computation in kernels

```
1  hpx::for_each_n(execution::par, ... , [](int i) {  
2      ...  
3  });  
4  
5  hpx::for_each_n(execution::par, ... , [](int i) {  
6      ...  
7  });  
8  
9  hpx::for_each_n(execution::par, ... , [](int i) {  
10     ...  
11  });  
12  
13  hpx::for_each_n(execution::par, ... , [](int i) {  
14     ...  
15  });
```

Prior Work

- Prior work: Not sufficient to simply use `hpx::for_each` constructs
- Well-balanced computation in kernels
- Requires additional optimizations to exploit parallelism

```
1  hpx::for_each_n(execution::par, ... , [](int i) {  
2      ...  
3  });  
4  
5  hpx::for_each_n(execution::par, ... , [](int i) {  
6      ...  
7  });  
8  
9  hpx::for_each_n(execution::par, ... , [](int i) {  
10     ...  
11  });  
12  
13  hpx::for_each_n(execution::par, ... , [](int i) {  
14     ...  
15  });
```

Manual Loop Partitioning

- Partition loop computation manually into tasks

```
1  std::vector// synchronization barrier
9
10 for (i = 0; i < N; i += P) {
11     auto f1 = hpx::async([i] (...) {
12         for (j = i; j < (i + P); j++) { ... }
13     });
14     v1.push_back(f1);
15 }
16 hpx::wait_all(v1); // synchronization barrier
```

Manual Loop Partitioning

- Partition loop computation manually into tasks
- Keep synchronization barrier
- No performance benefit at this point

```
1  std::vector<hpx::future<void>> v0, v1;
2  for (i = 0; i < N; i += P) {
3      auto f0 = hpx::async([i] (...) {
4          for (j = i; j < (i + P); j++) { ... }
5      });
6      v0.push_back(f0);
7  }
8  hpx::wait_all(v0); // synchronization barrier
9
10 for (i = 0; i < N; i += P) {
11     auto f1 = hpx::async([i] (...) {
12         for (j = i; j < (i + P); j++) { ... }
13     });
14     v1.push_back(f1);
15 }
16 hpx::wait_all(v1); // synchronization barrier
```

Independent Task Chains

- No dependencies between individual nodes in consecutive loops
- Global synchronization barrier not required for correctness

```
1  #pragma omp parallel for
2  for (i = 0; i < N; i++) {
3      xdd[i] = fx[i] * mass[i];
4  }
5
6  #pragma omp parallel for
7  for (i = 0; i < N; i++) {
8      xd[i] = xdd[i] * dt;
9  }
10
11 #pragma omp parallel for
12 for (i = 0; i < N; i++) {
13     x[i] = xd[i] * dt;
14 }
```

Independent Task Chains

- No dependencies between individual nodes in consecutive loops
- Global synchronization barrier not required for correctness
- Example: Calculation of node acceleration, velocity and position

```
1  #pragma omp parallel for
2  for (i = 0; i < N; i++) {
3      xdd[i] = fx[i] * mass[i];
4  }
5
6  #pragma omp parallel for
7  for (i = 0; i < N; i++) {
8      xd[i] = xdd[i] * dt;
9  }
10
11 #pragma omp parallel for
12 for (i = 0; i < N; i++) {
13     x[i] = xd[i] * dt;
14 }
```

Independent Task Chains

- Chain consecutive tasks with continuations

```
1  std::vector<hpx::future<void>> v;
2  for (i = 0; i < N; i += P) {
3      auto f1 = hpx::async([i] (...) {
4          for (j = i; j < (i + P); j++) { ... }
5      }).then([i] (auto &&f) {
6          for (j = i; j < (i + P); j++) { ... }
7      }).then([i] (auto &&f) {
8          for (j = i; j < (i + P); j++) { ... }
9      }).then([i] (auto &&f) {
10         for (j = i; j < (i + P); j++) { ... }
11     });
12     v.push_back(f1);
13 }
14 // synchronization barrier
15 auto f_all = hpx::when_all(v);
```

Independent Task Chains

- Chain consecutive tasks with continuations
- Some barriers are still required (e.g. between node- and element-wise calculations)
- Reduce to seven barriers in total

```
1  std::vector<hpx::future<void>> v;  
2  for (i = 0; i < N; i += P) {  
3      auto f1 = hpx::async([i] (...) {  
4          for (j = i; j < (i + P); j++) { ... }  
5      }).then([i] (auto &&f) {  
6          for (j = i; j < (i + P); j++) { ... }  
7      }).then([i] (auto &&f) {  
8          for (j = i; j < (i + P); j++) { ... }  
9      }).then([i] (auto &&f) {  
10         for (j = i; j < (i + P); j++) { ... }  
11     });  
12     v.push_back(f1);  
13 }  
14 // synchronization barrier  
15 auto f_all = hpx::when_all(v);
```

Independent Task Chains

- Chain consecutive tasks with continuations
- Some barriers are still required (e.g. between node- and element-wise calculations)
- Reduce to seven barriers in total
- Use `when_all()` for barriers

```
1  std::vector<hpx::future<void>> v;  
2  for (i = 0; i < N; i += P) {  
3      auto f1 = hpx::async([i] (...) {  
4          for (j = i; j < (i + P); j++) { ... }  
5      }).then([i] (auto &&f) {  
6          for (j = i; j < (i + P); j++) { ... }  
7      }).then([i] (auto &&f) {  
8          for (j = i; j < (i + P); j++) { ... }  
9      }).then([i] (auto &&f) {  
10         for (j = i; j < (i + P); j++) { ... }  
11     });  
12     v.push_back(f1);  
13 }  
14 // synchronization barrier  
15 auto f_all = hpx::when_all(v);
```

Combine Consecutive Loops

- Some loops contain little work

```
1  std::vector<hpx::future<void>> v;
2  for (i = 0; i < N; i += P) {
3      auto f1 = hpx::async([i] (...) {
4          for (j = i; j < (i + P); j++) { ... }
5          for (j = i; j < (i + P); j++) { ... }
6      }).then([i] (auto &&f) {
7          for (j = i; j < (i + P); j++) { ... }
8          for (j = i; j < (i + P); j++) { ... }
9      });
10     v.push_back(f1);
11 }
12 // synchronization barrier
13 auto f_all = hpx::when_all(v);
```

Combine Consecutive Loops

- Some loops contain little work
- Combining consecutive loops in one task
- Lower number of total tasks reduces scheduling overhead

```
1  std::vector<hpx::future<void>> v;
2  for (i = 0; i < N; i += P) {
3      auto f1 = hpx::async([i] (...) {
4          for (j = i; j < (i + P); j++) { ... }
5          for (j = i; j < (i + P); j++) { ... }
6      }).then([i] (auto &&f) {
7          for (j = i; j < (i + P); j++) { ... }
8          for (j = i; j < (i + P); j++) { ... }
9      });
10     v.push_back(f1);
11 }
12 // synchronization barrier
13 auto f_all = hpx::when_all(v);
```

Run Tasks In Parallel

- In some cases consecutive loops are independent (e.g. when calculating different nodal forces)

```
1  std::vector<hpx::future<void>> v;  
2  for (i = 0; i < N; i += P) {  
3      auto f1 = hpx::async([i] (...) {  
4          for (j = i; j < (i + P); j++) { ... }  
5          for (j = i; j < (i + P); j++) { ... }  
6      });  
7      v.push_back(f1);  
8      auto f2 = hpx::async([i] (...) {  
9          for (j = i; j < (i + P); j++) { ... }  
10         for (j = i; j < (i + P); j++) { ... }  
11     });  
12     v.push_back(f2);  
13 }  
14 // synchronization barrier  
15 auto f_all = hpx::when_all(v);
```

Run Tasks In Parallel

- In some cases consecutive loops are independent (e.g. when calculating different nodal forces)
- Allow scheduling of tasks in parallel

```
1  std::vector<hpx::future<void>> v;  
2  for (i = 0; i < N; i += P) {  
3      auto f1 = hpx::async([i] (...) {  
4          for (j = i; j < (i + P); j++) { ... }  
5          for (j = i; j < (i + P); j++) { ... }  
6      });  
7      v.push_back(f1);  
8      auto f2 = hpx::async([i] (...) {  
9          for (j = i; j < (i + P); j++) { ... }  
10         for (j = i; j < (i + P); j++) { ... }  
11     });  
12     v.push_back(f2);  
13 }  
14 // synchronization barrier  
15 auto f_all = hpx::when_all(v);
```

Evaluation: Methodology

- Compare runtimes of task-based HPX implementation to OpenMP-based reference
- Vary number of execution threads, problem size and material regions

Evaluation: Methodology

- Compare runtimes of task-based HPX implementation to OpenMP-based reference
- Vary number of execution threads, problem size and material regions
- Utilization of worker threads

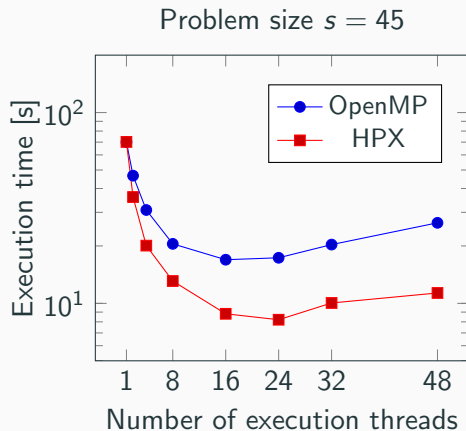
Evaluation: Methodology

- Compare runtimes of task-based HPX implementation to OpenMP-based reference
- Vary number of execution threads, problem size and material regions
- Utilization of worker threads
- Programming effort

Evaluation: Methodology

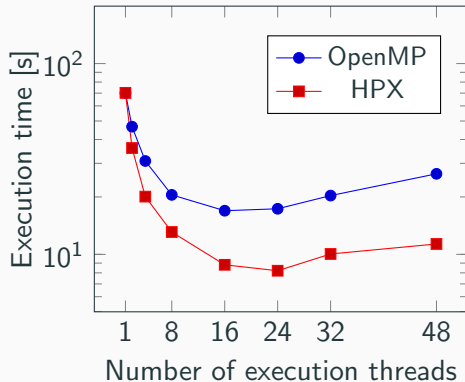
- Compare runtimes of task-based HPX implementation to OpenMP-based reference
- Vary number of execution threads, problem size and material regions
- Utilization of worker threads
- Programming effort
- System setup:
 - AMD EPYC 7443 24-core processor
 - HPX v1.10 with JEMalloc
 - Default scheduling policy
 - GCC 13.1.0

Evaluation: Runtime

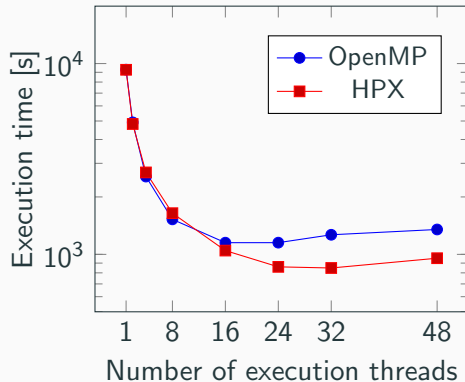


Evaluation: Runtime

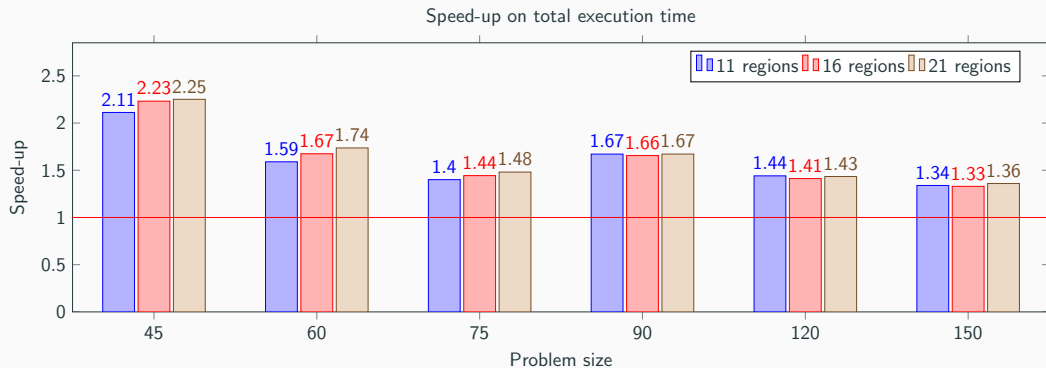
Problem size $s = 45$



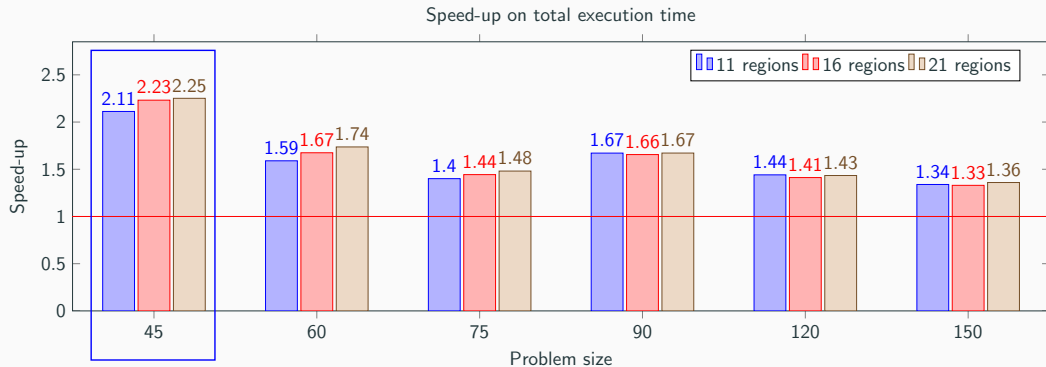
Problem size $s = 150$



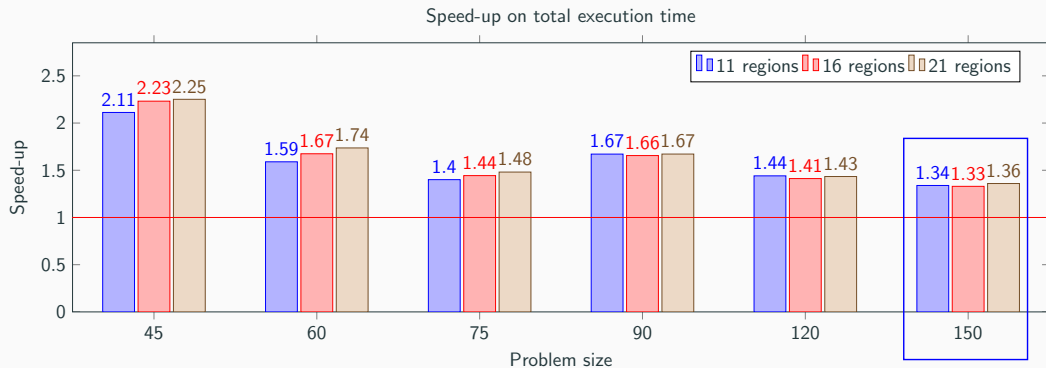
Evaluation: Speed-Up



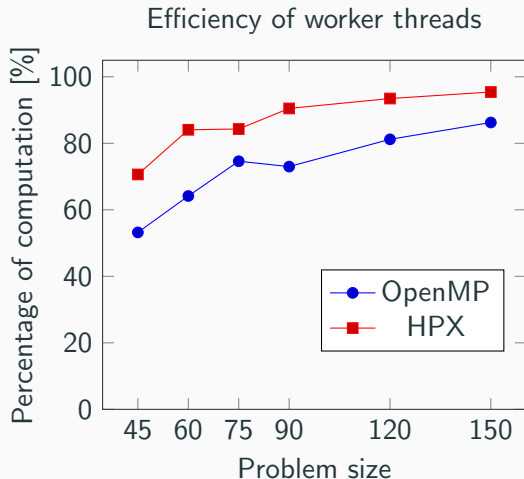
Evaluation: Speed-Up



Evaluation: Speed-Up

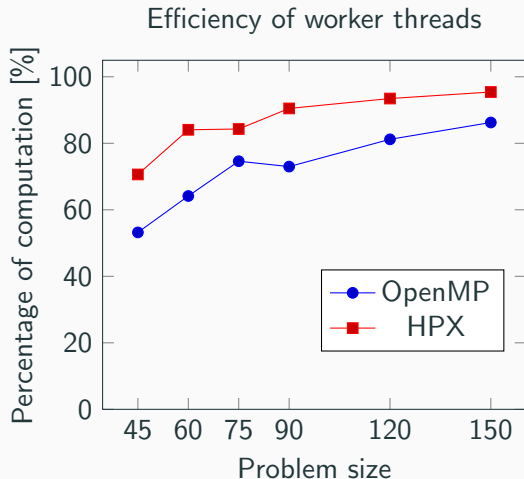


Evaluation: Efficiency



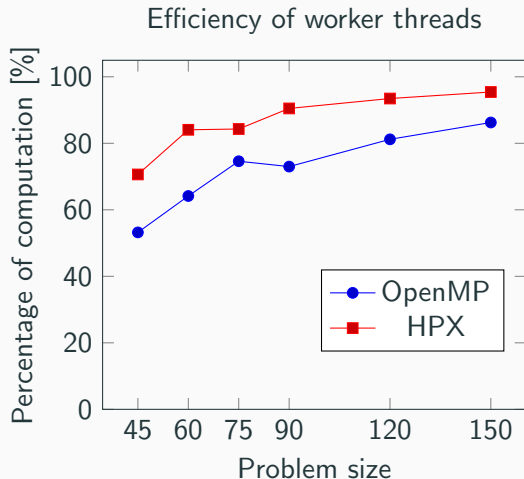
- Percentage of total runtime during which worker threads actually perform computations
 - Parallel regions (OpenMP)
 - Executing tasks (HPX)

Evaluation: Efficiency



- Percentage of total runtime during which worker threads actually perform computations
 - Parallel regions (OpenMP)
 - Executing tasks (HPX)
- Direct correlation to achieved speedups

Evaluation: Efficiency



- Percentage of total runtime during which worker threads actually perform computations
 - Parallel regions (OpenMP)
 - Executing tasks (HPX)
- Direct correlation to achieved speedups
- Increasing efficiency of OpenMP leads to decreasing speedups for larger problem sizes

Evaluation: Programming Effort

- Important aspect but hard to measure

Evaluation: Programming Effort

- Important aspect but hard to measure
- Parallelizing using OpenMP quite straight-forward

Evaluation: Programming Effort

- Important aspect but hard to measure
- Parallelizing using OpenMP quite straight-forward
- Building task-graph in HPX requires more in-depth analysis of data dependencies
- Necessary to exploit parallelism beyond loop granularity

Evaluation: Programming Effort

- Important aspect but hard to measure
- Parallelizing using OpenMP quite straight-forward
- Building task-graph in HPX requires more in-depth analysis of data dependencies
- Necessary to exploit parallelism beyond loop granularity
- Only minimal changes to code performing actual calculations
- Increased code complexity in main loop to create tasks

- Task-based implementation of the LULESH proxy application using HPX

- Task-based implementation of the LULESH proxy application using HPX
- Detailed overview of optimization steps

- Task-based implementation of the LULESH proxy application using HPX
- Detailed overview of optimization steps
- Achieved speedups between 1.33x and 2.25x
- Improved utilization of worker threads

- Task-based implementation of the LULESH proxy application using HPX
- Detailed overview of optimization steps
- Achieved speedups between 1.33x and 2.25x
- Improved utilization of worker threads
- Qualitative assessment of programming effort



Our implementation is available on Github



<https://github.com/esa-tu-darmstadt/lulesh-task-based-hpx>

Get in touch: kalkhof@esa.tu-darmstadt.de